

LOW LEVEL DESIGN (LLD)

RAG-BASED CUSTOMER SUPPORT ASSISTANT

1. MODULE LEVEL DESIGN

The system is designed in a modular format where each component performs a specific task in the RAG pipeline.

1. Document Processing Module

Responsibility:

Handles loading and preprocessing of PDF documents.

Functions:

- Load PDF using PyPDFLoader
- Extract raw text from pages
- Pass text to chunking module

Input:

- PDF file

Output:

- List of document pages (text format)

2. Chunking Module

Responsibility:

Splits large documents into smaller meaningful chunks.

Implementation:

- Uses CharacterTextSplitter
- Chunk size: 100–300 characters
- Overlap: 50 characters

Purpose:

- Maintains context continuity
- Improves retrieval accuracy

Output:

- List of document chunks

3. Embedding Module**Responsibility:**

Converts text chunks into numerical vector representations.

Model Used:

- Sentence Transformers (HuggingFace)

Output:

- Dense vector embeddings for each chunk

4. Vector Storage Module (ChromaDB)**Responsibility:**

Stores embeddings and enables similarity search.

Features:

- Stores vector + metadata
- Supports fast nearest neighbor search
- Persistent storage capability

Input:

- Embedded chunks

Output:

- Indexed vector database

5. Retrieval Module**Responsibility:**

Finds relevant document chunks based on user query.

Process:

- Convert query into embedding
- Compare with stored vectors
- Retrieve top-K similar chunks

Algorithm:

- Cosine similarity / vector distance

Output:

- Relevant document chunks

6. Response Generation Module**Responsibility:**

Generates final answer using retrieved context.

Logic:

- Combine retrieved chunks
- Apply rule-based generation (or LLM in advanced version)
- Construct response

Output:

- Natural language answer
- Confidence score

7. Workflow Engine (LangGraph Module)**Responsibility:**

Controls execution flow of system.

Nodes:

- Retrieve Node
- Generate Node
- HITL Node
- Output Node

Flow:

retrieve → generate → decision → HITL/output

8. Human-in-the-Loop (HITL) Module**Responsibility:**

Handles uncertain or low-confidence responses.

Trigger Condition:

- $\text{confidence} < \text{threshold} (0.7)$

Process:

- System pauses execution
- Human agent provides response
- Final answer is updated

2. DATA STRUCTURES

State Object (Core System State)

```
State = {  
  "query": str,  
  "docs": List[str],  
  "answer": str,  
  "confidence": float,  
  "needs_hitl": bool  
}
```

Document Structure

- page_content: raw text
- metadata: page number, source

Chunk Structure

- text chunk
- chunk_id
- embedding vector

Query Response Structure

- query
- retrieved context
- generated answer

- confidence score

3. WORKFLOW DESIGN (LANGGRAPH)

Nodes:

1. retrieve()
 - Fetches relevant chunks
2. generate()
 - Creates response from context
3. hitl()
 - Escalates to human if needed
4. output()
 - Displays final response

Edges:

- retrieve → generate
- generate → (hitl OR output)
- hitl → output

4. CONDITIONAL ROUTING LOGIC

Decision Rule:

If confidence < 0.7 → HITL

Else → Direct output

Conditions:

- Low confidence response
- Insufficient context
- Ambiguous query

5. HITL DESIGN

Trigger:

- Confidence below threshold

Flow:

1. System pauses execution
2. Query sent to human agent
3. Human provides response
4. System overrides AI response

6. API DESIGN (LOGICAL)

Input:

User Query (string)

Output:

Answer

Confidence Score

Source (AI / Human)

7. ERROR HANDLING

- No documents found → "Limited info available"
- Retrieval failure → fallback response
- Empty context → HITL triggered
- System failure → safe default output